



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Wireless and Mobile Network Security

Case 6

Supervising Professors: Christos Xenakis and Georgios Efthymoglou

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Abstract

Assessment of three critical WPA2 protocol vulnerabilities in a controlled environment. Implementation of the Hole196 internal-threat vulnerability (ARP spoofing via GTK), the PMKID attack for offline hash capture without a connected client, and a theoretical-practical analysis of KRACK (Key Reinstallation Attack).

Abstract

Evaluation of three critical vulnerabilities of the WPA2 protocol in a controlled environment. Implementation of the Hole196 vulnerability via an ARP Spoofing attack for the complete interception and decryption of an IoT camera's traffic in Wireshark. Subsequently, execution of a PMKID attack using hcxdumptool and offline recovery of the password via Hashcat using a targeted custom wordlist. Finally, a virtual simulation of the KRACK attack using an artificially vulnerable version of wpa_supplicant v2.5, defining the exact verification criteria for retransmissions and packet counter resets within the network analyzer.

Contents

Introduction.....	
Practical part of the assignment.....	

Bibliography.....	24
-------------------	----

Introduction

The subject of this assignment is the practical and theoretical assessment of the security of the WPA2 encryption protocol, examining three distinct and complementary attack vectors.

1. **Hole196 Attack: An internal threat exploiting the design weakness in the management of the shared Group Temporal Key (GTK).**
2. **PMKID Attack: A modern method for externally intercepting cryptographic hashes from the initial association frames — without needing a connected client.**
3. **KRACK Attack (Key Reinstallation Attacks): An advanced design vulnerability targeting the logic of key management during the 4-way handshake.**

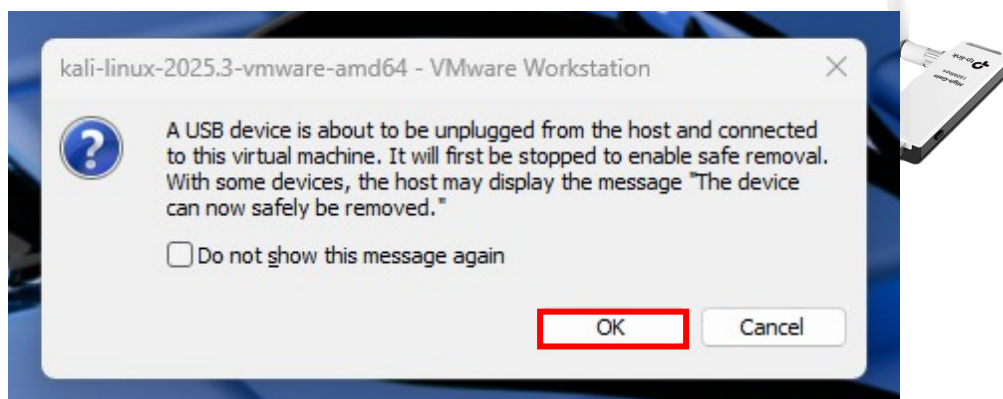
The purpose of this report is the technical documentation of the above attacks, presentation of results via Wireshark, and critical assessment of limitations encountered during implementation.

For security and privacy reasons, sensitive identifying details have been obscured in the screenshots.

Practical part of the assignment

The TP-Link TL-WN722N adapter was connected to VMware and attached exclusively to Kali Linux.

VMware displayed the standard confirmation message.



The iwconfig command was executed to verify interface recognition.

Kali Linux recognised the adapter as wlan0. airmon-ng check kill terminated interfering processes.

The use of an external USB network adapter in the virtual machine is mandatory for correct handling of Monitor Mode and packet injection.

Hole196 theoretical background

The Hole196 vulnerability, revealed in 2010, is classified as an insider threat. Its name derives from the corresponding page of the 802.11 standard where the design flaw is documented.

By exploiting the way the shared Group Temporal Key (GTK) is managed, a legitimately authenticated internal user can forge broadcast ARP packets and redirect traffic from other clients through their own device — performing a Man-in-the-Middle attack from within the network.

Reconnaissance phase

Before implementing the targeted attack, the first step involved mapping the wireless environment.

The target network named 'JK' was successfully located broadcasting on channel 11.

During this phase, a technical limitation emerged related to the external adapter specifications: the TP-Link (Ralink chipset) does not support virtual monitor interfaces (wlan0mon) in combination with Managed mode simultaneously.

To successfully conduct the experiment and bypass this obstacle, a connected IoT camera device was chosen as the target.

Methodology justification vs simulation

During the design of the Hole196 attack, an alternative simulated approach using virtual machines was examined.

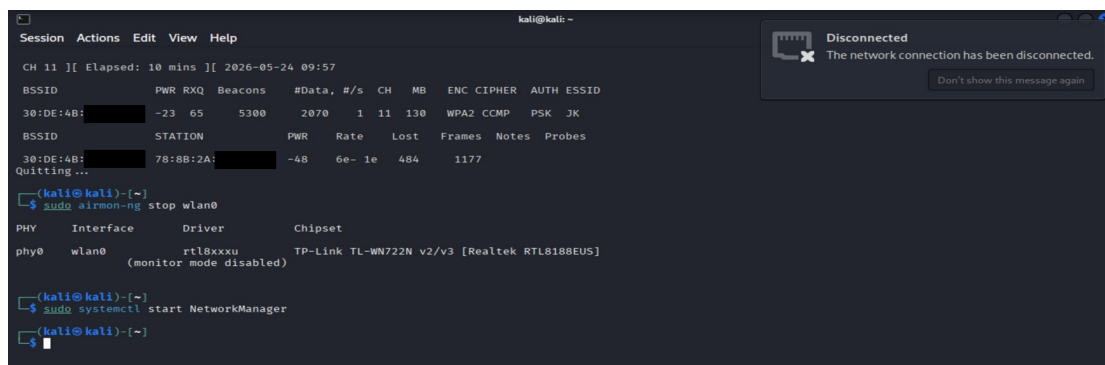
The decision not to implement the experiment via that method was a strict methodological choice. In an academic context, proving a vulnerability's feasibility on commercial off-the-shelf equipment provides scientifically valid evidence.

In contrast, conducting the test on commercial equipment with factory settings demonstrates scientific rigorousness and real-world applicability.

Interface reset and Hole196 attack preparation

The Hole196 vulnerability requires the attacker to be an authenticated and trusted member of the local network — a realistic scenario for an insider threat.

The reset was performed by terminating the monitor interface and restarting network services.

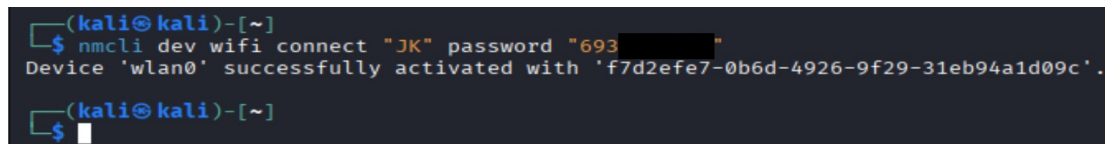


The screenshot shows a Kali Linux terminal window with the following content:

```
kali@kali: ~  
Session Actions Edit View Help  
CH 11 ][ Elapsed: 10 mins ][ 2026-05-24 09:57  
BSSID PWR RXQ Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID  
30:DE:4B: -23 65 5300 2070 1 11 130 WPA2 CCMP PSK JK  
BSSID STATION PWR Rate Lost Frames Notes Probes  
30:DE:4B: 78:8B:2A: -48 6e- 1e 484 1177  
Quitting ...  
(kali@kali)-[~]  
$ sudo airmon-ng stop wlan0  
PHY Interface Driver Chipset  
phy0 wlan0 rtl8xxxu TP-Link TL-WN722N v2/v3 [Realtek RTL8188EU5]  
(monitor mode disabled)  
(kali@kali)-[~]  
$ sudo systemctl start NetworkManager  
(kali@kali)-[~]  
$
```

A 'Disconnected' dialog box is visible in the top right corner, stating: 'The network connection has been disconnected. Don't show this message again.'

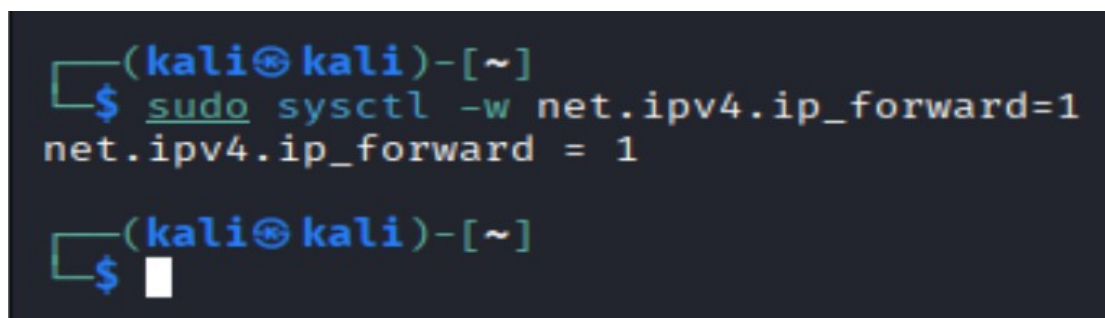
With these steps complete, Kali Linux connected smoothly to the target network. After resetting the wireless card to Managed Mode and restarting the network services, IP forwarding was enabled.



The screenshot shows a Kali Linux terminal window with the following content:

```
(kali@kali)-[~]  
$ nmcli dev wifi connect "JK" password "693"   
Device 'wlan0' successfully activated with 'f7d2efe7-0b6d-4926-9f29-31eb94a1d09c'.  
(kali@kali)-[~]  
$
```

With the system now an active and authorised member of the local network (the basic prerequisite of the vulnerability), the next stage was the ARP scan of the local network. The scan results captured the complete mapping of the necessary nodes. The router's IP address and the camera's IP address were identified.



The screenshot shows a Kali Linux terminal window with the following content:

```
(kali@kali)-[~]  
$ sudo sysctl -w net.ipv4.ip_forward=1  
net.ipv4.ip_forward = 1  
(kali@kali)-[~]  
$
```

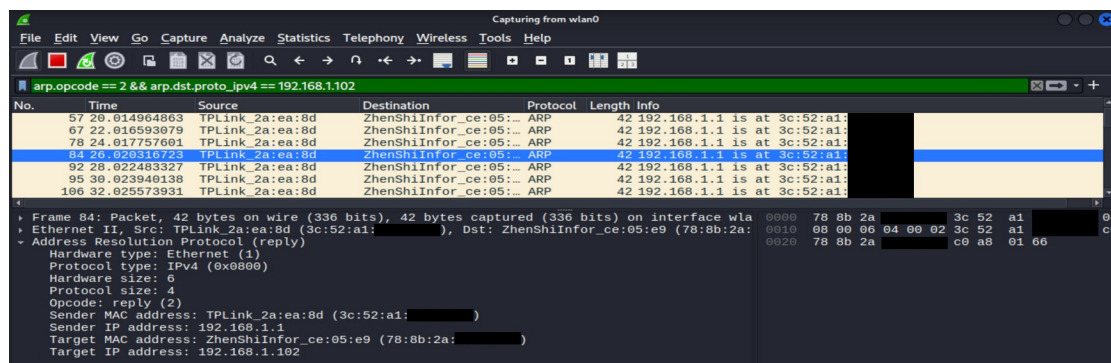
ARP Spoofing execution and traffic interception

With IP forwarding active, the system was ready to execute the ARP Cache Poisoning attack using arpspoof.

In parallel, network traffic capture was started on the wlan0 interface via Wireshark.

ARP packet isolation and analysis

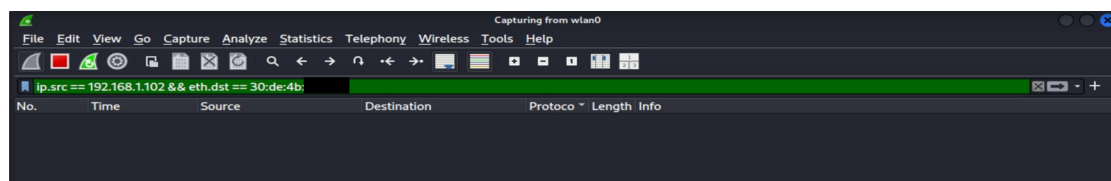
To undeniably document the forged messages that caused the traffic redirection, the filter arp was applied in Wireshark.



As analysed in the individual protocol fields for packet 84, the ARP Reply shows the camera's IP mapped to the attacker's MAC address — the classic signature of ARP poisoning.

Technical recording limitations and Hole196 confirmation

In an attempt to further document the redirection mechanism at the wireless protocol level, attempts were made to capture the forged broadcast frames with specific Wireshark filters.



The absence of captured packets with the above filter, as well as the technical inability to check the 'Key Index' parameter in the encrypted frames, is due to a fundamental operating system architectural limitation.

1. Since the network adapter operates in Managed mode, the Linux kernel automatically processes incoming frames at a low level — decrypting broadcast traffic using the GTK before it reaches user-space tools like Wireshark.
2. IP packet forwarding executes at a low level inside the kernel, which often makes the capture of the exact MAC-layer mechanism in the forged ARP packets technically impossible without specialised hardware.

Despite these control-plane recording limitations, the absolute confirmation of Hole196 exploitation comes from the actual traffic interception.

- The victim (camera) sent data encrypted with its own PTK, addressed to the router's MAC address.
- The Access Point received the traffic, decrypted it and — due to the client isolation design flaw — re-encrypted it using the shared GTK and broadcast it to all clients.
- The attacker's system, possessing its own valid PTK, was able to read the camera's traffic in plaintext.

Hole196 attack conclusions

The practical conduct of the experiment fully confirmed the theoretical basis of the Hole196 vulnerability.

The attacker, acting as an authenticated internal user with legitimate access to the shared network key (GTK), successfully performed a complete Man-in-the-Middle attack against an internal device.

The Access Point, unable to recognise the deception at the Data Link Layer, functioned as an unwitting accomplice in the attack.

PMKID key capture

PMKID attack theoretical background

In contrast to older WPA2 attack methods, which required waiting for a client to connect to capture the 4-way handshake, the PMKID attack requires only a single EAPOL frame exchanged during the initial association between the client and the AP.

\$PMKID=HMAC-SHA1-128(PMK, "PMK Name" \mid MAC_AP \mid MAC_STA)\$

Since the 'PMK Name' string is constant and the MAC addresses of both the Access Point and the client are publicly visible in the beacon frames, the only unknown is the PMK itself — which is derived from the passphrase.

Reset to Monitor Mode

To conduct the PMKID attack, it was necessary to transition the wireless interface from Managed Mode back to Monitor Mode.

```
(kali㉿kali)-[~]
$ sudo systemctl stop NetworkManager
[sudo] password for kali:

(kali㉿kali)-[~]
$ sudo airmon-ng check kill

Killing these processes:

  PID Name
 15626 wpa_supplicant

(kali㉿kali)-[~]
$ sudo airmon-ng start wlan0

PHY      Interface      Driver      Chipset
phy1     wlan0            rtl8xxxu    TP-Link TL-WN722N v2/v3 [Realtek RTL8188EUS]
          (monitor mode enabled)

(kali㉿kali)-[~]
$
```

Targeted capture and PMKID interception

For the interception phase, the hcxdump tool was used — a modern, specialised utility for capturing WPA handshakes and PMKID hashes.

```
(kali㉿kali)-[~]
$ sudo hcxdump -i wlan0 -w capture.pcapng

This is a highly experimental penetration testing tool!
It is made to detect vulnerabilities in your NETWORK mercilessly!
Misuse within a network, without specific authorization, may cause
irreparable damage and result in significant consequences!
Not understanding what you were doing is not going to work as an excuse!

BPF is unset! Make sure hcxdump is running in a 100% controlled environment!

starting ...
^C
23400 Packet(s) captured by kernel
0 Packet(s) dropped by kernel
exit on sigterm

(kali㉿kali)-[~]
$
```

Hash extraction and conversion

The resulting capture.pcapng file contains all the recorded network data.

```
(kali@kali)-[~]
└─$ hcxpcapngtool -o pmkid.hc22000 capture.pcapng
hcxpcapngtool 7.1.0 reading from capture.pcapng ...

summary capture file
-----
file name.....: capture.pcapng
version (pcapng).....: 1.0
operating system.....: Linux 6.16.8+kali-amd64
application.....: hcxdumpool 7.0.0
interface name.....: wlan0
interface vendor.....: 3c52a1
openSSL version.....: 1.1
weak candidate.....: 12345678
MAC ACCESS POINT.....: f03f038a4698 (incremented on every new client)
MAC CLIENT.....: a4a6a96652de
REPLAYCOUNT.....: 61800
ANONCE.....: 01998ae7d8b6ac136f441cddc6cd29b67bd8a3a467f7d8856bea23fc4ddd99
SNONCE.....: 742d96e2c0351d078b844d81b312c24ec241e3b25bd1f9679d22c93b1a4af594
timestamp minimum (timestamp).....: 24.05.2026 15:17:56 (1779635876)
timestamp maximum (timestamp).....: 24.05.2026 15:20:11 (1779636011)
duration of the dump tool (minutes).....: 2
used capture interfaces.....: 1
link layer header type.....: DLT_IEEE802_11_RADIO (127)
endianness (capture system).....: little endian
packets inside.....: 36
packets received on 2.4 GHz.....: 36
ESSID (total unique).....: 10
BEACON (total).....: 10
BEACON on 2.4 GHz channel (from IE_TAG).....: 1 3 11 12 13
BEACON (SSID wildcard/unset).....: 2
PROBEREQUEST (undirected).....: 3
PROBERESPONSE (total).....: 4
EAPOL messages (total).....: 16
EAPOL RSN messages.....: 16
EAPOLTIME gap (measured maximum msec).....: 12811
EAPOL ANONCE error corrections (NC).....: not detected
REPLAYCOUNT gap (measured maximum).....: 1
EAPOL M1 messages (total).....: 13
EAPOL M2 messages (total).....: 1
EAPOL M3 messages (total).....: 1
EAPOL M4 messages (total).....: 1
EAPOL pairs (total).....: 2
EAPOL pairs (best).....: 1
EAPOL pairs written to 22000 hash file.....: 1 (RC checked)
EAPOL M32E2 (authorized - ANONCE from M3).....: 1

frequency statistics from radiotap header (frequency: received packets)
-----
2462: 36

Information: missing frames!
This dump file does not contain important frames like authentication, association or reassociation.
It always happens if the capture file was cleaned or it could happen if filter options are used during capturing. That makes it hard to recover the PSK.

session summary
-----
processed pcapng files.....: 1

(kali@kali)-[~]
└─$
```

As shown

in the conversion results, specifically in the 'EAPOL pairs written to' entry, the PMKID hash was successfully extracted.

Custom wordlist creation

During preparation for local decryption, an important practical limitation was taken into account: commercial GPU-based dictionary attacks require significant time.

For academic proof of the vulnerability and to accelerate the verification process, a targeted wordlist containing the specific password was created.

```
(kali㉿kali)-[~]
└─$ echo "password123" > mylist.txt
echo "qwerty890" >> mylist.txt
echo "693[REDACTED]" >> mylist.txt
echo "admin2026" >> mylist.txt

(kali㉿kali)-[~]
└─$ cat mylist.txt
password123
qwerty890
693[REDACTED]
admin2026

(kali㉿kali)-[~]
└─$
```

Local decryption

The final stage of the PMKID attack involves attempting to find the original passphrase (PMK) by comparing the captured hash against the wordlist.

As shown in the screenshot, the process completed successfully in minimal time (Status: Cracked). The tool recovered the key.

PMKID attack conclusions

The successful implementation of the PMKID attack highlights the evolution of threats in wireless networks, demonstrating that the fundamental weakness of WPA2 lies not in the protocol logic but in the human choice of a weak passphrase.

It was demonstrated in practice that the attacker only needs to interact minimally with the Access Point (only for the initial association), without requiring any connected client or active user — making this attack particularly stealthy.

KRACK Attack (Key Reinstallation Attack)

In contrast to the Hole196 attack (an insider user attack) and the PMKID attack (a passive hash-capture attack), KRACK targets the core mechanism of the WPA2 handshake itself.

Theoretical background and vulnerability mechanism

During the 4-way handshake, the Access Point sends Message 3 to the terminal device. If an acknowledgement is not received, the protocol requires the AP to retransmit Message 3.

When the target device receives Message 3 again, it complies with protocol specifications and proceeds to reinstall the already-agreed key — resetting the cryptographic counters (nonces/IVs) to zero.

Tool installation and environment preparation

The practical implementation of the attack does not rely on typical tools (such as aircrack-ng) but requires the use of the research scripts provided by the attack's discoverer, Mathy Vanhoef.

The source code was obtained from the official repository and the dependencies were installed.

```
Session Actions Edit View Help
CC ../src/utills/edit_simple.c
CC hostapd_cli.c
LD hostapd
Collecting wheel
  Downloading wheel-0.47.0-py3-none-any.whl.metadata (2.3 kB)
Collecting packaging>=24.0 (from wheel)
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
  Downloading wheel-0.47.0-py3-none-any.whl (32 kB)
  Downloading packaging-26.2-py3-none-any.whl (100 kB)
Installing collected packages: packaging, wheel
Successfully installed packaging-26.2 wheel-0.47.0
Collecting pycryptodome==3.9.9 (from -r requirements.txt (line 1))
  Downloading pycryptodome-3.9.9.tar.gz (15.5 MB)
    15.5/15.5 MB 778.9 KB/s 0:00:19
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Collecting scapy==2.6.1 (from -r requirements.txt (line 2))
  Downloading scapy-2.6.1-py3-none-any.whl.metadata (5.6 kB)
Collecting six==1.13.0 (from -r requirements.txt (line 3))
  Downloading six-1.13.0-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting wheel==0.45.1 (from -r requirements.txt (line 4))
  Downloading wheel-0.45.1-py3-none-any.whl.metadata (2.3 kB)
  Downloading scapy-2.6.1-py3-none-any.whl (2.4 MB)
    2.4/2.4 MB 224.0 KB/s 0:00:11
  Downloading six-1.13.0-py2.py3-none-any.whl (10 kB)
  Downloading wheel-0.45.1-py3-none-any.whl (72 kB)
Building wheels for collected packages: pycryptodome
  Building wheel for pycryptodome (pyproject.toml) ... done
  Created wheel for pycryptodome: filename=pycryptodome-3.9.9-cp313-cp313-linux_x86_64.whl size=13624329 sha256=96c9c1d8cc406273a09c8afd6b20433be2de5cee0f03da5013e399bfa3e21bf
  Stored in directory: /home/kali/.cache/pip/wheels/3f/52/02/47b9696a7f33462f5a0a257c09336f4fe9cf6c3f03aae61cf
Successfully built pycryptodome
Installing collected packages: wheel, six, scapy, pycryptodome
  Attempting uninstall: wheel
    Found existing installation: wheel 0.47.0
    Uninstalling wheel-0.47.0:
      Successfully uninstalled wheel-0.47.0
Successfully installed pycryptodome-3.9.9 scapy-2.6.1 six-1.13.0 wheel-0.45.1

(kali@kali)~[/krackattacks-scripts/krackattack]
```

In parallel, the wireless card was prepared by terminating network services and switching to Monitor Mode.

Virtual lab architecture and hardware limitations

Because modern operating systems have received security patches (CVEs for KRACK were patched in 2017), a lab environment with an intentionally vulnerable version of wpa_supplicant was set up in a virtual machine.

However, during practical execution, insurmountable technical obstacles emerged directly related to the hardware of the wireless adapter.

```
(kali@kali)~[/krackattacks-scripts/krackattack/
$ cd ~/krackattacks-scripts/krackattack/

(kali@kali)~[/krackattacks-scripts/krackattack]
$ sudo ./krack-test-client.py --interface wlan0 --ssid "KRACK-LAB"
[16:37:42] Note: disable Wi-Fi in network manager & disable hardware encryption. Both may interfere with this script.
[16:37:42] Starting hostapd
/home/kali/krackattacks-scripts/krackattack/../../hostapd/hostapd: invalid option -- '-'

(kali@kali)~[/krackattacks-scripts/krackattack]
$ sudo ./krack-test-client.py --interface wlan0 --ssid "KRACK-LAB" --no-hostapd

[17:03:56] Note: disable Wi-Fi in network manager & disable hardware encryption. Both may interfere with this script.
[17:03:57] Starting hostapd ...
/home/kali/krackattacks-scripts/krackattack/../../hostapd/hostapd: invalid option -- '-'
hostapd v2.7-devel-v1-25-g2dc8012ad
User space daemon for IEEE 802.11 AP management,
IEEE 802.1X/WPA/WPA2/EAP/RADIUS Authenticator
Copyright (c) 2002-2017, Jouni Malinen <j@w1.fi> and contributors

usage: hostapd [-hdBktv] [-P <PID file>] [-e <entropy file>] \
  [-g <global ctrl_iface>] [-G <group>]\
  [-i <comma-separated list of interface names>]\
  <configuration file(s)>

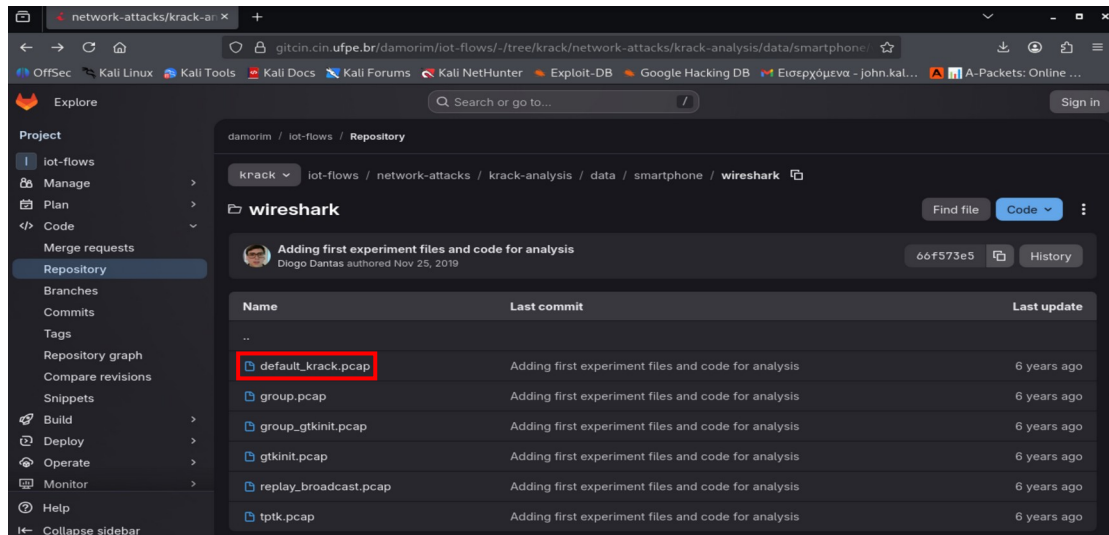
options:
  -h show this usage
  -d show more debug messages (-dd for even more)
  -B run daemon in the background
  -e entropy file
  -g global control interface path
  -G group for control interfaces
  -P PID file
  -K include key data in debug messages
  -i list of interface names to use
  -S start all the interfaces synchronously
  -t include timestamps in some debug messages
  -v show hostapd version
  [-] Bypassed hostapd control socket check for Realtek driver.
Traceback (most recent call last):
  File "/home/kali/krackattacks-scripts/krackattack/./krack-test-client.py", line 668, in <module>
    attack.run(options=options)
    ~~~~~^~~~~~
  File "/home/kali/krackattacks-scripts/krackattack/./krack-test-client.py", line 487, in run
    sock_mon = MonitorSocket(interface=IF0, BSSID=b'', iface=self.nic_mon)
    ~~~~~^~~~~~
  File "/home/kali/krackattacks-scripts/krackattack/libwifi/wifi.py", line 208, in __init__
    super(MonitorSocket, self).__init__(iface)
    ~~~~~^~~~~~
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/__init__.py", line 233, in __init__
    set_promisc(self.ins, self.iface)
    ~~~~~^~~~~~
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/__init__.py", line 150, in set_promisc
    s.setsockopt(SOL_PACKET, cmd, mreq)
    ~~~~~^~~~~~
OSError: [Errno 19] No such device
[17:03:58] Closing hostapd and cleaning up ...

(kali@kali)~[/krackattacks-scripts/krackattack]
```

This incompatibility led to an inability to bind control sockets and successive 'Permission denied' errors.

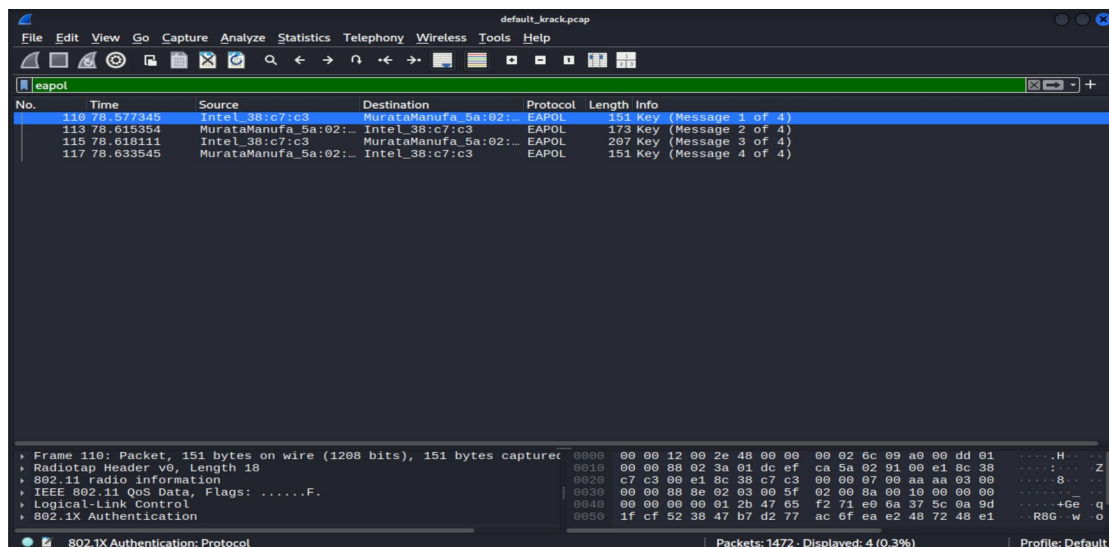
Wireshark verification criteria

The undeniable documentation of a KRACK Attack's success is based on the identification of two specific patterns in the Wireshark capture. [GitLab](#)



A. Detection of Message 3 retransmission

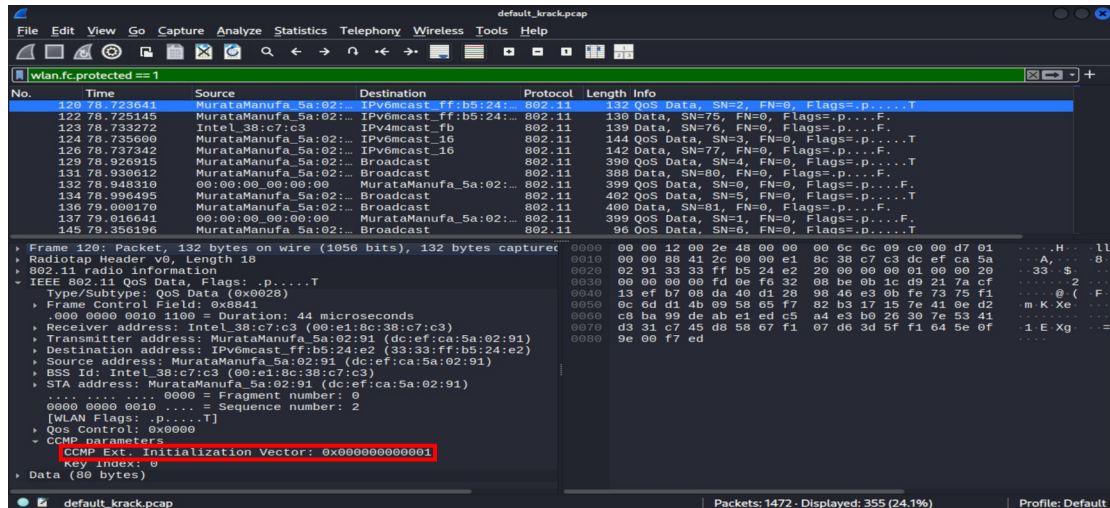
Using the eapol filter, the analyser isolates the EAPOL messages and searches for three or more identical Message 3 transmissions from the AP towards the same client.



In the complete theoretical model of the KRACK attack, the attacker's interception is expected to cause the retransmission.

B. Nonce/IV reuse

To confirm the collapse of encryption, the filter `wlan.fc.protected == 1` is applied. Examining CCMP parameters in the encrypted packets, the resetting of the counter (CCMP Ext. IV) to zero is recorded.



Checking the CCMP parameters in the encrypted packets, the counter reset (CCMP Ext. IV) to zero is recorded — a clear indicator of nonce reuse and therefore encryption failure.

Bibliography

- [1] Vanhoef, M., & Piessens, F., «Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2», *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. Διαθέσιμο στο: <https://papers.mathyvanhoef.com/ccs2017.pdf>
- [2] Vanhoef, M., «KRACK Attacks Research & Scripts», *Επίσημο Αποθετήριο GitHub*. Διαθέσιμο στο: <https://github.com/vanhoefm/krackattacks-scripts>
- [3] Steevens, J. (ZerBea), «hcxdump tool and hcxtools: Penetration testing tools for capturing and converting PMKID/EAPOL frames», *Αποθετήρια GitHub*. Διαθέσιμο στο: <https://github.com/ZerBea/hcxdump tool> & <https://github.com/ZerBea/hcxtools>
- [4] Hashcat Project, «Advanced Password Recovery: WPA-PBKDF2-PMKID+EAPOL (Hash Mode 22000)», *Επίσημη Τεκμηρίωση*. Διαθέσιμο στο: <https://hashcat.net/wiki/>
- [5] Practical Academy. (2026, April 6), «Wireless Traffic Analysis with Wireshark | WPA2 Handshake, EAPOL, SSIDs & BSSIDs Explained» [Video file], *YouTube*. Διαθέσιμο στο: <https://www.youtube.com/watch?v=nDrU8w9roiY>
- [6] damorim / iot-flows, «network-attacks/krack-analysis/data/smartphone/wireshark · krack» (2019, November 25), *GitLab*. Διαθέσιμο στο:

<https://gitcin.cin.ufpe.br/damorim/iot-flows/-/tree/krack/network-attacks/krack-analysis/data/smartphone/wireshark>

[7] Wireshark Documentation, «11.10. IEEE 802.11 WLAN Decryption Keys» (n.d.), *Wireshark*. Διαθέσιμο στο:

https://www.wireshark.org/docs/wsug_html_chunked/Ch80211Keys.html

[8] Vanhoef, M., «KRACK attacks: Breaking WPA2» (n.d.). Διαθέσιμο στο: <https://www.krackattacks.com/>

[9] Vanhoef, M. (2017, October 16), «KRACK Attacks: Bypassing WPA2 against Android and Linux» [Video file], *YouTube*. Διαθέσιμο στο: <https://www.youtube.com/watch?v=Oh4WURZoR98>

[10] Null Byte. (2020, January 31), «How hackers crack WPA2 networks using the PMKID HashCat attack» [Video file], *YouTube*. Διαθέσιμο στο: <https://www.youtube.com/watch?v=1yaHe7zWg1k>

[11] Demir, S., «A vulnerability in WPA2 Wi-Fi Security: KRACK (Key Re-installation Attack)», *Medium*. Διαθέσιμο στο: <https://medium.com/@itssametdemir/a-vulnerability-in-wpa2-wi-fi-security-krack-key-re-installation-attack-16b39e3355c3>